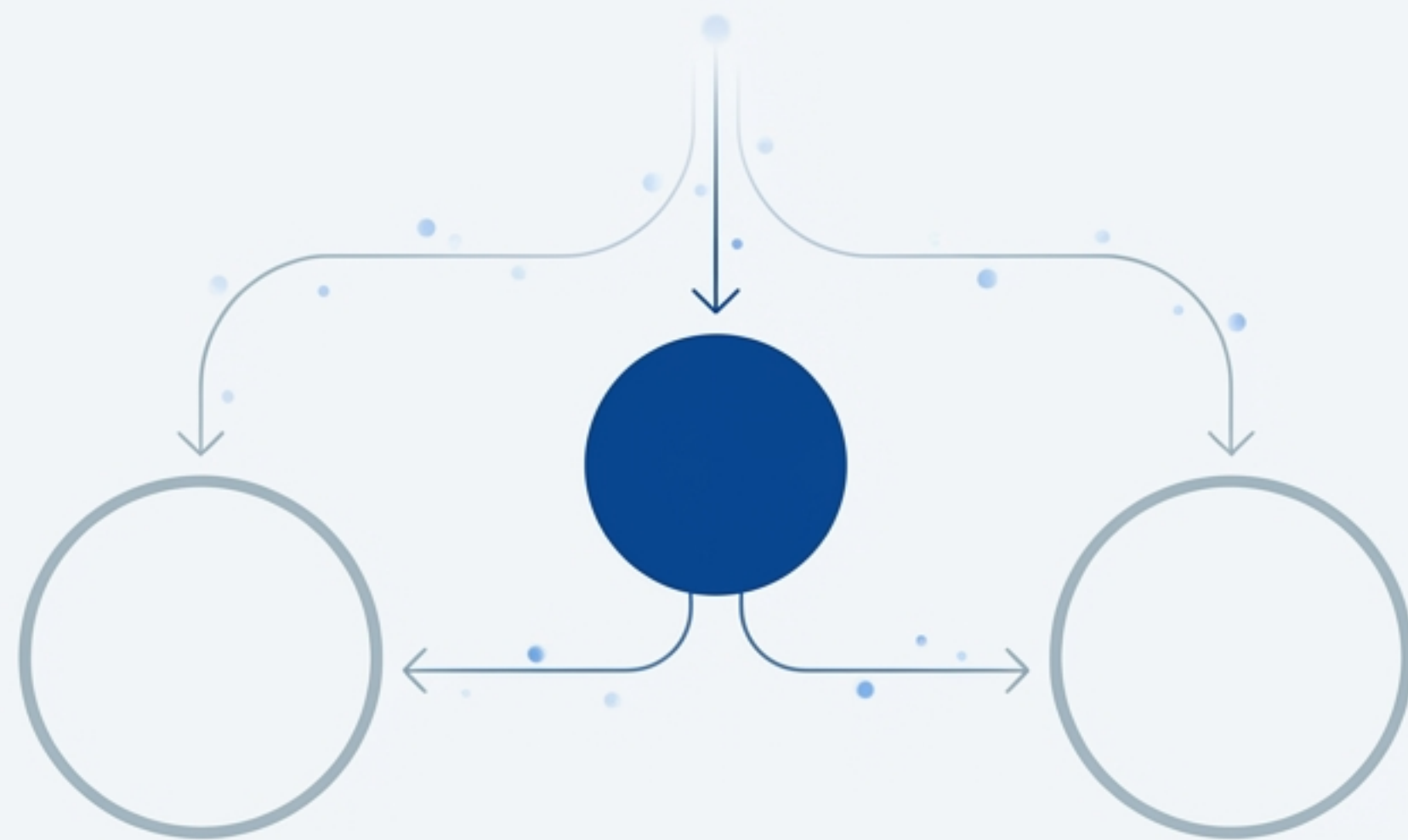


LangGraph 子图可控性

构建模块化、可复用的智能代理



本次分享大纲

1.  问题的提出：复杂工作流的“单体”困境

2.  核心概念解析：像微服务一样构建LangGraph应用

3.  实战演练（上）：构建“故障分析”独立子图

3.  核心概念解析：像微服务一样构建LangGraph应用

4.  实战演练（下）：构建“成功报告”独立子图

5.  最终组装：创建“总控图”实现智能路由

6.  总结

复杂工作流的困境：当所有逻辑都纠缠在一起

单体（Monolithic）Graph 应用难以维护、复用和测试。

难以维护：一个节点的改动可能引发全局性的连锁反应。

复用性差：无法将特定功能（如“报告生成”）轻松移植到其他项目中。

测试困难：无法对单一逻辑路径进行隔离测试，调试过程复杂。



LangGraph的答案：像微服务一样构建代理

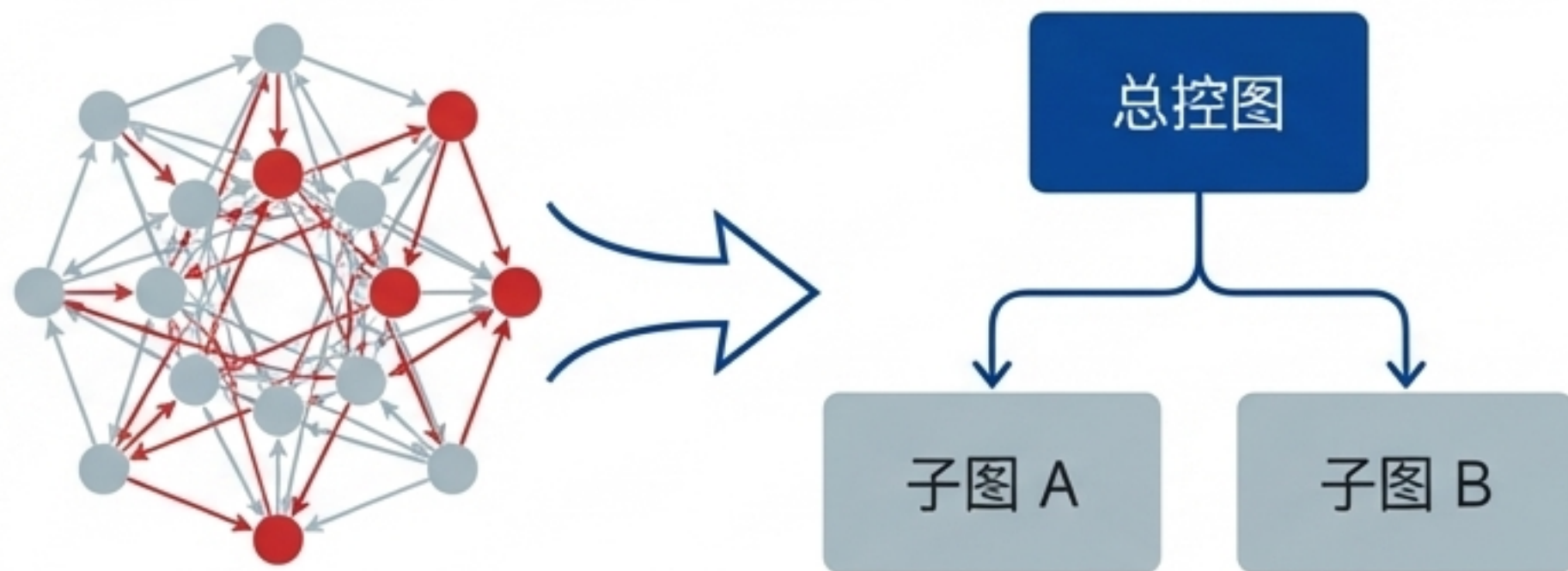
通过子图（Sub-graphs）实现“分而治之”。

- **子图（Subgraph）**

一个完全独立的、可执行的图，拥有自己的状态和逻辑。可以看作一个“功能单元”或“微服务”。

- **总控图（Controller Graph）**

一个“父”图，负责根据输入和条件，将任务“路由”到合适的子图。

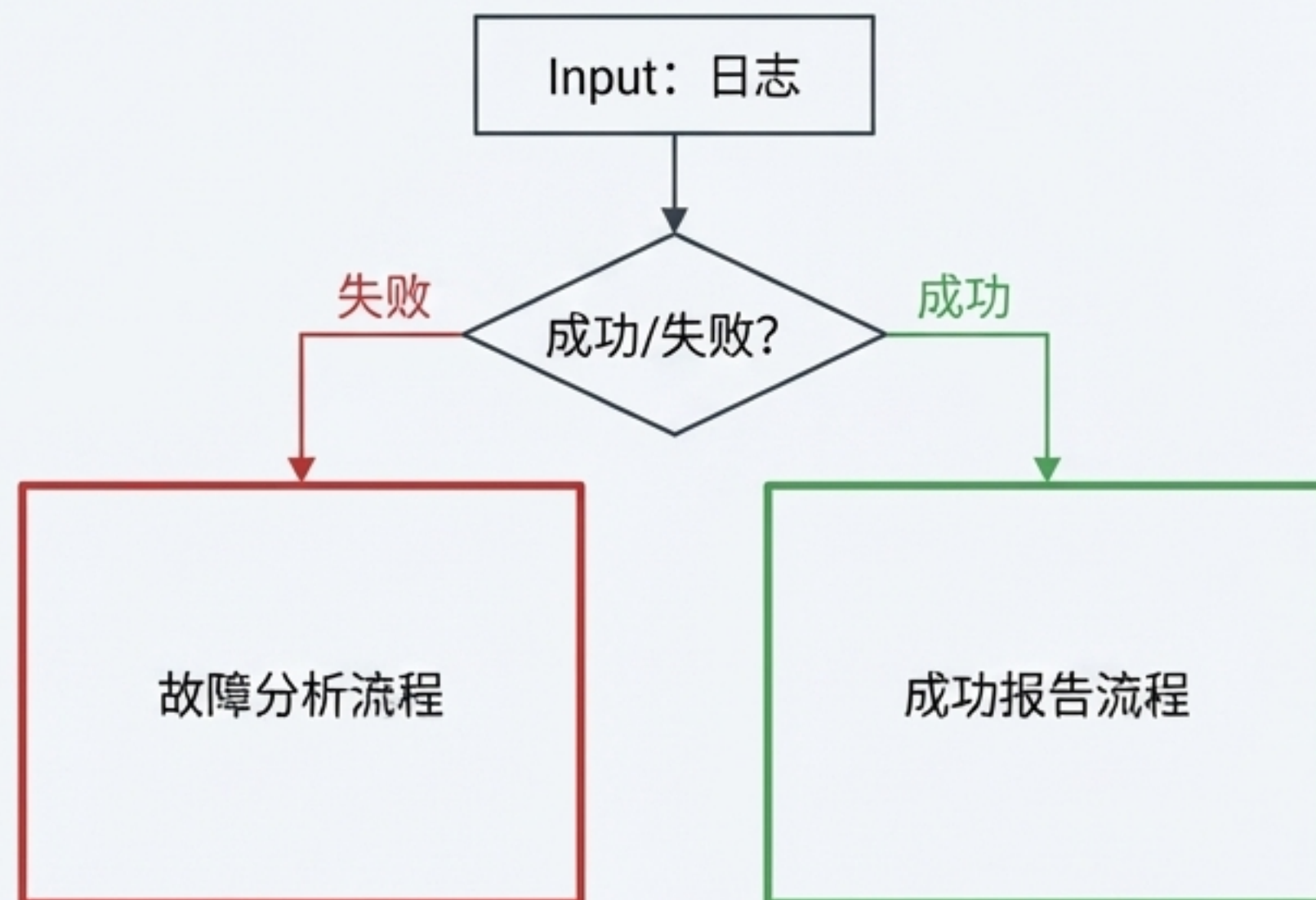


实战场景：智能日志分析与报告系统

目标：自动处理程序运行日志，并根据日志内容（成功或失败）执行不同的处理流程。

核心逻辑

1. 接收原始日志（raw_logs）。
2. 判断日志类型。
3. 失败日志：路由到【故障分析子图】进行深度分析和总结。
4. 成功日志：路由到【成功报告子图】生成摘要并发送通知。



故障分析子图（1/2）：定义“状态”与“节点”

每个子图都是一个独立的世界，我们首先定义它的“记忆”（状态）和“能力”（节点）

```
1 # 定义故障分析状态的结构
2 class FailureAnalysisState(TypedDict):
3     docs: List[Logs]      # 日志列表
4     failures: List[Logs]  # 失败的日志列表
5     fa_summary: str       # 故障分析总结
6
7 # 获取失败数据的节点
8 def get_failures(state):
9     # ... implementation ...
10
11 # 生成故障分析总结的节点
12 def generate_summary(state):
13     # ... implementation ...
```



故障分析子图（2/2）：组装、编译与可视化

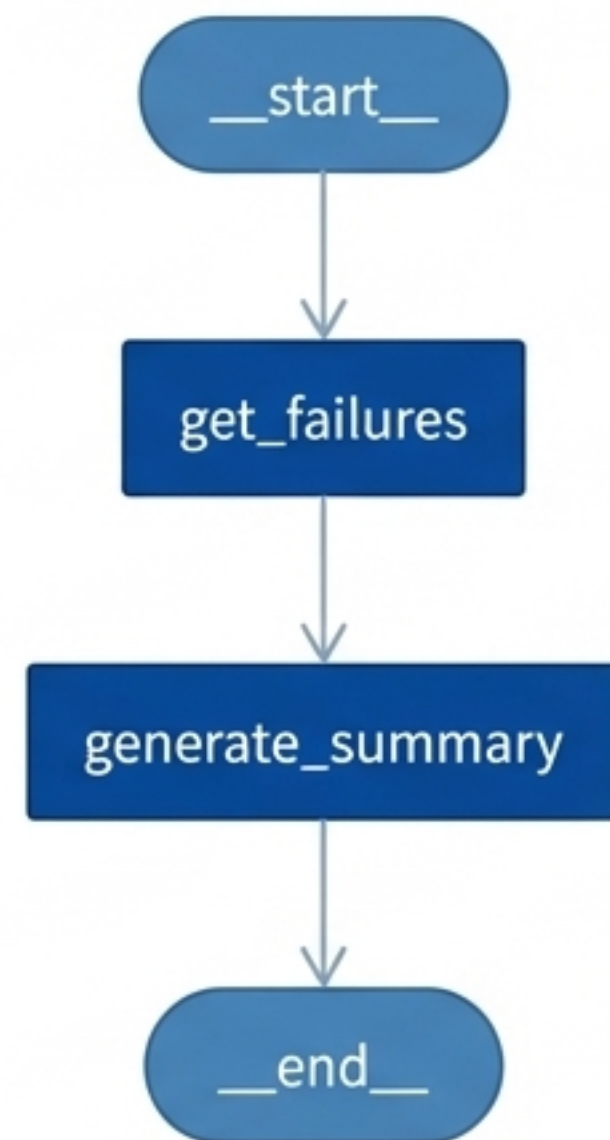
将节点和边连接起来，最终编译成一个可执行、可复用的组件。

```
# 创建故障分析状态图
fa_builder = StateGraph(FailureAnalysisState)

# 添加节点
fa_builder.add_node("get_failures", get_failures)
fa_builder.add_node("generate_summary", generate_summary)

# 添加边，定义流程
fa_builder.add_edge(START, "get_failures")
fa_builder.add_edge("get_failures", "generate_summary")
fa_builder.add_edge("generate_summary", END)

# 编译图
fa_graph = fa_builder.compile()
```



成功报告子图：构建另一个独立功能单元

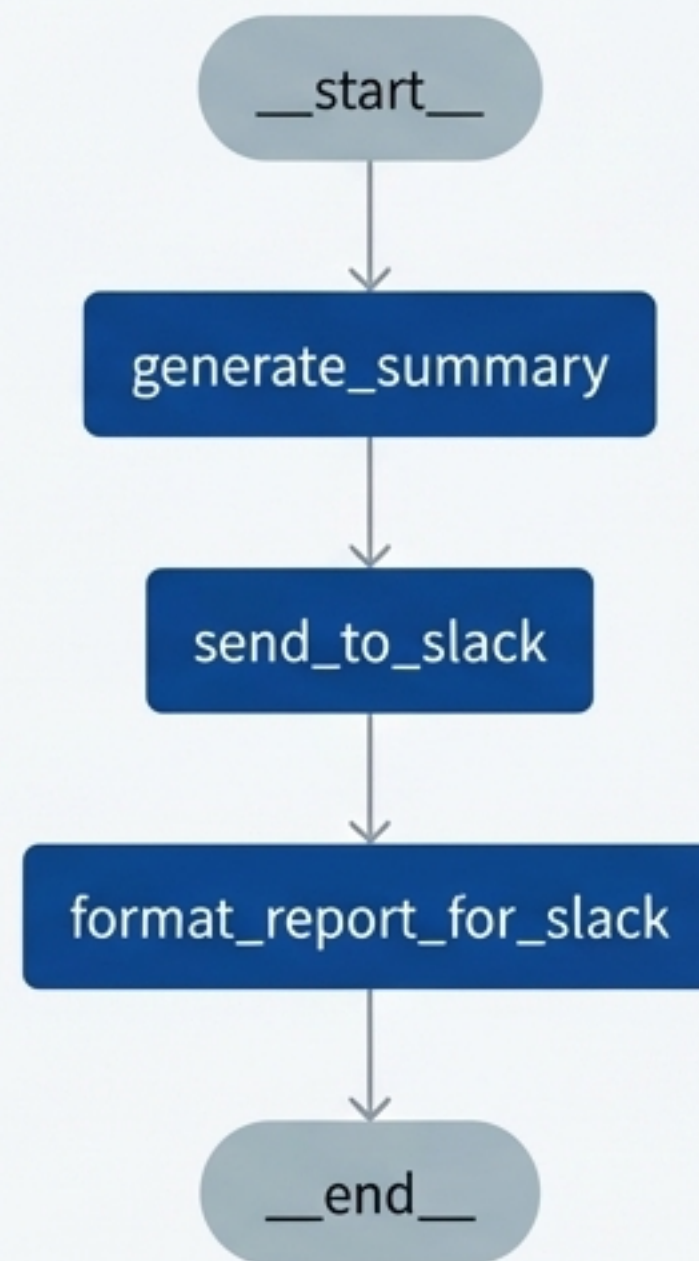
采用完全相同的模式，我们可以构建一个处理成功日志并发送Slack通知的独立流程。

State Definition

```
class QuestionSummarizationState(TypedDict):  
    docs: List[Logs]  
    qs_summary: str  
    report: str
```

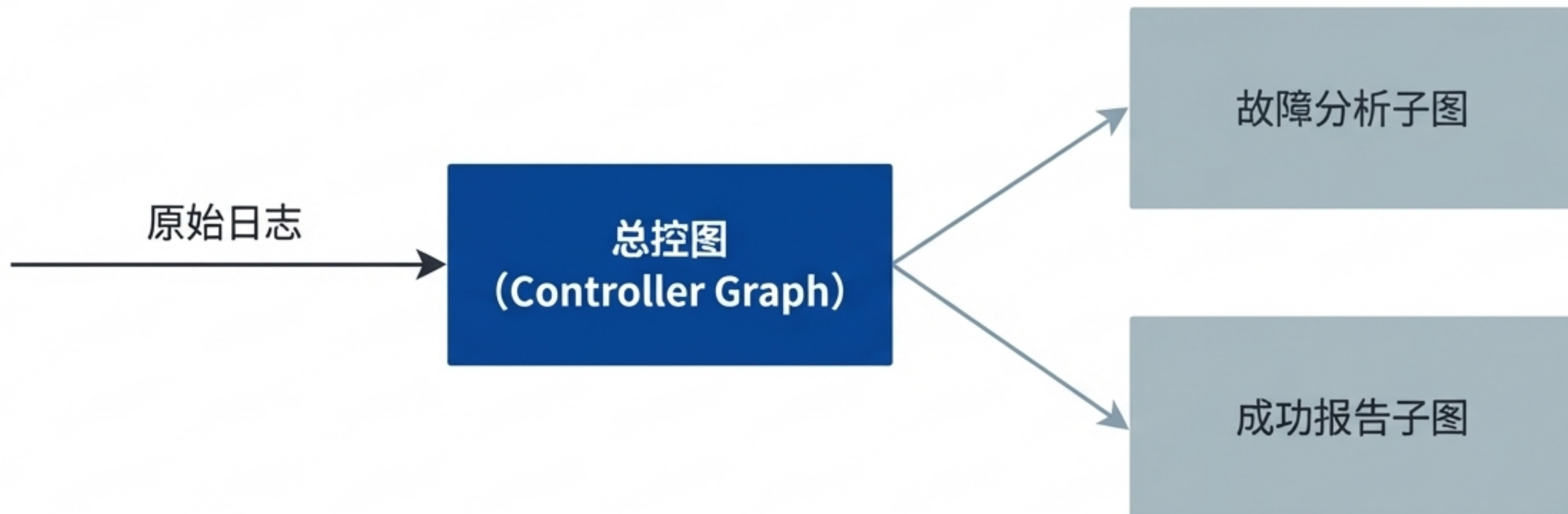
Nodes

- `generate\_summary`
- `send\_to\_slack`
- `format\_report\_for\_slack`



最终组装：创建“总控图”进行调度

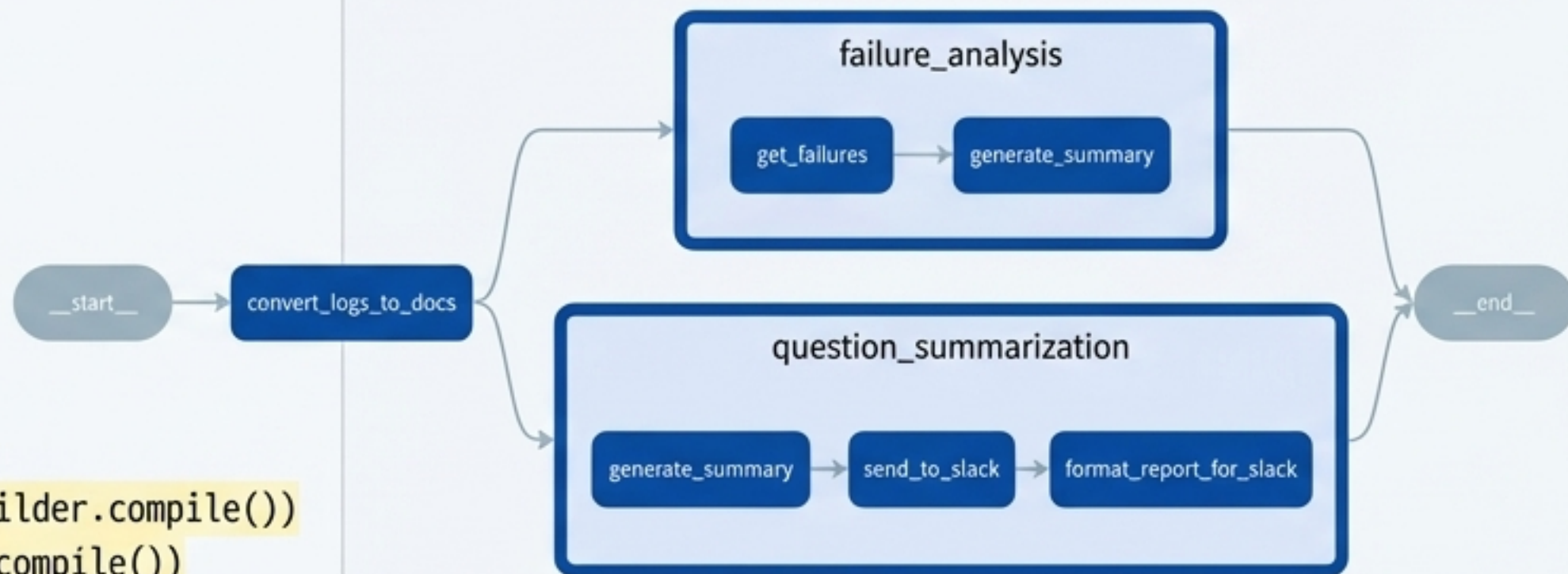
总控图不执行具体的业务逻辑，它的核心职责是接收初始数据，并将其“路由”到正确的子图。



总控图实现（1/2）：将“图”作为“节点”

这是子图可控性的核心魔法：将编译好的子图（`fa\_graph`、`qs\_graph`）本身作为节点添加到父图中。

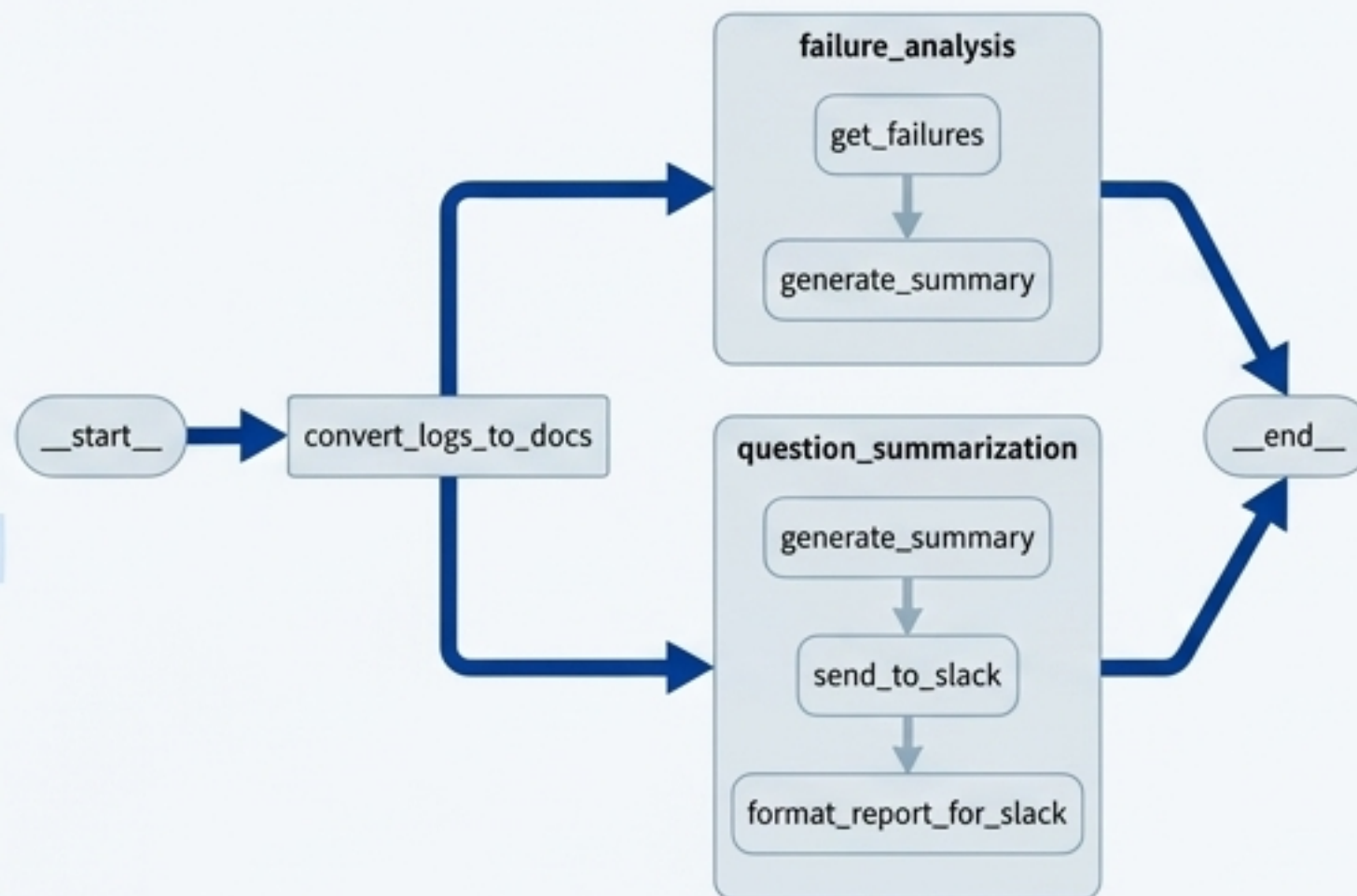
```
1 # 定义总控图的状态
2 class EntryGraphState(TypedDict):
3     raw_logs: Annotated[list, add]
4     # ... other state fields
5
6 entry_builder = StateGraph(EntryGraphState)
7
8 # ... 添加预处理节点 ...
9
10 # 将编译好的子图作为节点添加到总控图中
11 entry_builder.add_node("question_summarization", qs_builder.compile())
12 entry_builder.add_node("failure_analysis", fa_builder.compile())
```



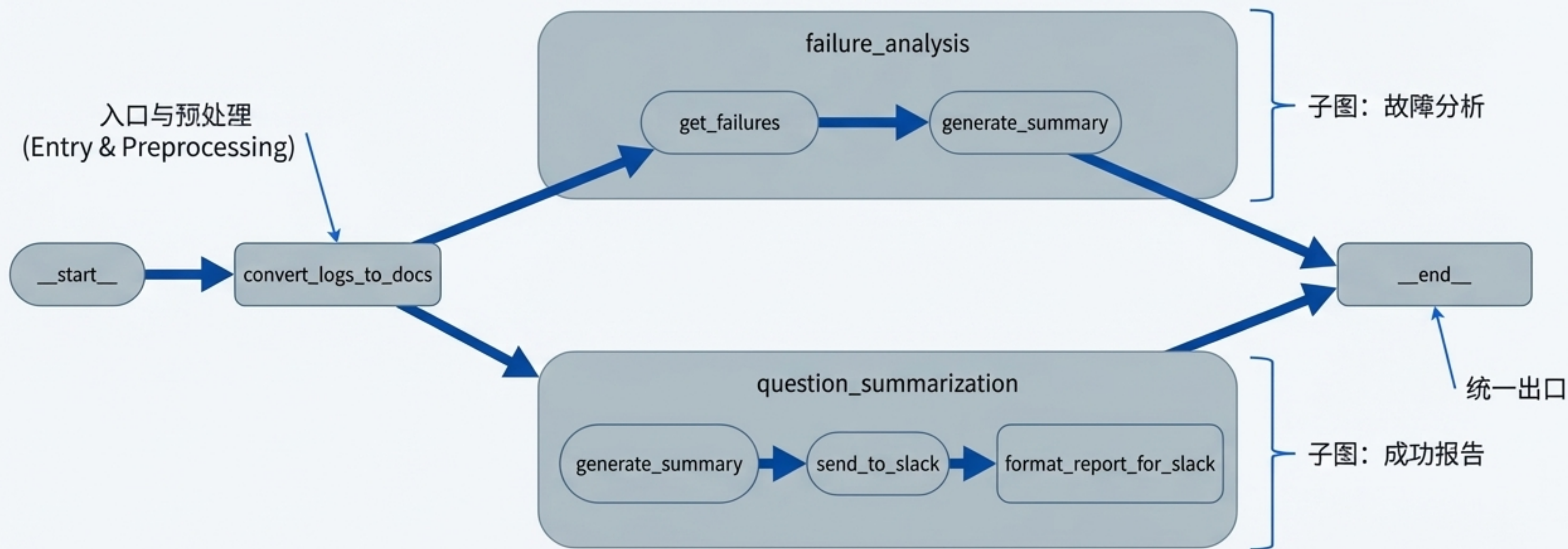
总控图实现（2/2）：定义路由与数据流

使用 `add\_edge` 来控制数据如何从预处理节点（`convert\_logs\_to\_docs`）流向各个子图。

```
1 # 添加预处理节点
2 entry_builder.add_node("convert_logs_to_docs", convert_logs_to_docs)
3
4 # 定义数据流：从入口流向预处理节点
5 entry_builder.add_edge(START, "convert_logs_to_docs")
6
7 # 定义路由：从预处理节点流向两个子图
8 entry_builder.add_edge("convert_logs_to_docs", "failure_analysis")
9 entry_builder.add_edge("convert_logs_to_docs", "question_summarization")
10
11 # 将子图连接到统一的出口
12 entry_builder.add_edge("failure_analysis", END)
13 entry_builder.add_edge("question_summarization", END)
```



最终成果：清晰、模块化、可控的工作流



核心要点回顾



模块化

将复杂流程分解为独立的子图，让逻辑更清晰。



可复用

编译后的子图是标准组件，可在任何工作流中被调用。



可控性

在父图中通过（条件）边，实现对子图的动态调用和路由。



易测试

每个子图都可以被独立测试，极大简化了开发和调试。

感谢观看
Q & A